



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE
LA RECHERCHE SCIENTIFIQUE ET DE
L'INNOVATION
UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES -
KHOURIBGA



TP3 : Book Reading tracker

avec TypeScript

Réalisé par :
TAFTAF Wijdane

Encadré par : Pr. Amal Ourdou

Année Universitaire : 2025 / 2026

Table des matières

Introduction	2
Spécifications du TP	3
0.1 Description fonctionnelle	3
0.2 Structure d'un livre	3
0.3 Contraintes sur les valeurs	3
Structure du Code	5
0.4 Organisation du projet	5
0.5 Implémentation du backend	6
0.5.1 Modèle Mongoose	6
0.5.2 Routes de l'API	7
0.5.3 Serveur Express	7
0.6 Implémentation du frontend	8
0.6.1 Interface utilisateur	8
0.6.2 Classe <code>Book</code> (module TypeScript)	8
0.6.3 Logique de la page	9
0.7 Tests et validation	10
0.7.1 Ajout d'un livre	10
0.7.2 Calcul du pourcentage et du statut	10
0.7.3 Suppression d'un livre	10
Conclusion	11

Introduction

Dans le cadre du module de **TypeScript et développement Web**, il nous a été demandé de réaliser une application complète permettant de suivre la lecture de livres : *Book Reading Tracker*.

L'objectif de ce travail pratique est double :

- Mettre en pratique les concepts fondamentaux de TypeScript : typage statique, classes, modules, énumérations.
- Concevoir une mini-application structurée suivant une architecture moderne : **frontend** (TypeScript + TailwindCSS) et **backend** (Node.js + TypeScript + MongoDB).

Ce rapport décrit de manière détaillée les choix techniques, la conception de la solution, ainsi que l'implémentation réalisée.

Spécifications du TP

Tp demandée doit répondre aux exigences suivantes :

0.1 Description fonctionnelle

Tp *Book Reading Tracker* doit permettre :

- L'enregistrement de nouveaux livres via un formulaire Web.
- L'affichage de la liste des livres enregistrés.
- Le calcul et l'affichage du pourcentage de lecture par livre.
- L'affichage d'un tableau de bord global :
 - nombre total de livres,
 - nombre de livres terminés,
 - total de pages lues.
- La suppression d'un livre.

0.2 Structure d'un livre

Chaque livre possède les attributs suivants :

- **title** : chaîne de caractères.
- **author** : chaîne de caractères.
- **pages** : nombre total de pages.
- **status** : statut de lecture (Enum).
- **price** : prix du livre.
- **pagesRead** : nombre de pages lues .
- **format** : format du livre .
- **suggestedBy** : personne ayant recommandé le livre.
- **finished** : booléen indiquant si le livre est terminé.

0.3 Contraintes sur les valeurs

Statut (Status)

- Read
- Re-read
- DNF (Did Not Finish)
- Currently reading
- Returned Unread
- Want to read

Format (Format)

- Print
- PDF
- Ebook
- AudioBook

Règle sur `finished` Par défaut, `finished = false`. Ce champ doit automatiquement passer à `true` lorsque :

$$pagesRead = pages$$

Structure du Code

Tp est organisée selon une architecture **client-serveur** :

- **Frontend** :
 - Une page HTML.
 - Stylée avec **TailwindCSS** pour un design moderne et dynamique.
 - Logique en **TypeScript** compilé en JavaScript.
 - Une classe **Book** définie dans son propre module.
- **Backend** :
 - **Node.js** + **Express** + **TypeScript**.
 - Connexion à une base de données **MongoDB**.
 - Exposition d'une API REST :
 - GET /api/books
 - POST /api/books
 - DELETE /api/books/:id
- **Base de données** :
 - Stockage persistant des livres avec **Mongoose**.

0.4 Organisation du projet

Listing 1 – Structure générale du projet

```
book-tracker/  
  backend/  
    src/  
      models/Book.ts  
      routes/books.ts  
      server.ts  
      tsconfig.json  
  frontend/  
    index.html  
    src/  
      Book.ts  
      main.ts  
      tsconfig.json  
    dist/
```

Cette séparation claire respecte les bonnes pratiques et facilite l'évolution de l'application.

0.5 Implémentation du backend

0.5.1 Modèle Mongoose

Le modèle MongoDB reprend strictement les spécifications du TP et utilise des énumérations TypeScript pour le statut et le format.

```
import { Schema, model, Document } from 'mongoose';

export enum Status {
  Read = 'Read',
  Reread = 'Re-read',
  DNF = 'DNF',
  CurrentlyReading = 'Currently reading',
  ReturnedUnread = 'Returned Unread',
  WantToRead = 'Want to read'
}

export enum Format {
  Print = 'Print',
  PDF = 'PDF',
  Ebook = 'Ebook',
  AudioBook = 'AudioBook'
}

export interface IBook extends Document {
  title: string;
  author: string;
  pages: number;
  status: Status;
  price: number;
  pagesRead: number;
  format: Format;
  suggestedBy: string;
  finished: boolean;
}
```

```
const bookSchema = new Schema<IBook>({
  title: { type: String, required: true },
  author: { type: String, required: true },
  pages: { type: Number, required: true },
  status: { type: String, enum: Object.values(Status), required: true },
  price: { type: Number, required: true },
  pagesRead: { type: Number, required: true },
  format: { type: String, enum: Object.values(Format), required: true },
  suggestedBy: { type: String, required: true },
  finished: { type: Boolean, default: false }
});

// Met finished = true si pagesRead >= pages
bookSchema.pre('save', function (next) {
  this.finished = this.pagesRead >= this.pages;
  next();
});

export const BookModel = model<IBook>('Book', bookSchema);
```

0.5.2 Routes de l'API

- GET /api/books : retourne la liste des livres.
- POST /api/books : ajoute un nouveau livre.
- DELETE /api/books/:id : supprime un livre.

```
import { Router } from 'express';
import { BookModel } from '../models/Book';

const router = Router();

// GET /api/books : liste tous les livres
router.get('/', async (_req, res) => {
  const books = await BookModel.find();
  res.json(books);
});

// POST /api/books : ajoute un livre
router.post('/', async (req, res) => {
  try {
    const book = new BookModel(req.body);
    await book.save();
    res.status(201).json(book);
  } catch (e: any) {
    res.status(400).json({ error: e.message });
  }
});

// DELETE /api/books/:id : supprime un livre
router.delete('/:id', async (req, res) => {
  await BookModel.findByIdAndDelete(req.params.id);
  res.status(204).end();
});

export default router;
```

0.5.3 Serveur Express

```
import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';
import booksRouter from './routes/books';

const app = express();
app.use(cors());
app.use(express.json());

// connexion MongoDB
mongoose.connect('mongodb://localhost:27017/book_tracker')
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error('Mongo error:', err));

// routes
app.use('/api/books', booksRouter);

const PORT = 3000;
app.listen(PORT, () => {
  console.log('Server running on http://localhost:${PORT}');
});
```

Cette implémentation valide la partie **persistance** et la communication avec le frontend.

0.6 Implémentation du frontend

0.6.1 Interface utilisateur

L'interface est réalisée en HTML et stylisée avec **TailwindCSS**. Elle comprend :

- Un formulaire pour saisir les informations du livre.
- Une section de statistiques globales.
- Une liste dynamique des livres avec affichage du pourcentage de lecture.
- Un bouton de suppression pour chaque livre.

0.6.2 Classe Book (module TypeScript)

Conformément au sujet, la classe Book est définie dans son propre module.

```
export enum Status {
  Read = 'Read',
  Reread = 'Re-read',
  DNF = 'DNF',
  CurrentlyReading = 'Currently reading',
  ReturnedUnread = 'Returned Unread',
  WantToRead = 'Want to read'
}

export enum Format {
  Print = 'Print',
  PDF = 'PDF',
  Ebook = 'Ebook',
  AudioBook = 'AudioBook'
}

export class Book {
  constructor(
    public _id: string | null,
    public title: string,
    public author: string,
    public pages: number,
    public status: Status,
    public price: number,
    public pagesRead: number,
    public format: Format,
    public suggestedBy: string,
    public finished: boolean = false
  ) {
    this.updateFinished();
  }
}
```

```
updateFinished(): void {
  this.finished = this.pagesRead >= this.pages;
}

currentlyAt(): string {
  const pct = (this.pagesRead / this.pages) * 100;
  return `${pct.toFixed(1)}%`;
}

async deleteBook(): Promise<void> {
  if (!this._id) return;
  await fetch(`http://localhost:3000/api/books/${this._id}`, {
    method: 'DELETE'
  });
}
```

0.6.3 Logique de la page

Le fichier `main.ts` se charge de :

- Récupérer la liste des livres depuis l'API.
- Instancier des objets `Book`.
- Mettre à jour le DOM (cartes de livres, statistiques).
- Gérer la soumission du formulaire et la suppression.

0.7 Tests et validation

Les principaux scénarios testés sont :

0.7.1 Ajout d'un livre

- Saisie des champs obligatoires.
- Vérification que `pagesRead ≤ pages`.
- Envoi d'une requête POST vers `/api/books`.
- Actualisation automatique de la liste.

0.7.2 Calcul du pourcentage et du statut

- Vérification de la méthode `currentlyAt()`.
- Validation du passage de `finished` à `true` lorsque `pagesRead = pages`.

0.7.3 Suppression d'un livre

- Clic sur le bouton *Delete*.
- Envoi d'une requête DELETE `/api/books/:id`.
- Mise à jour de l'affichage et de la base de données.

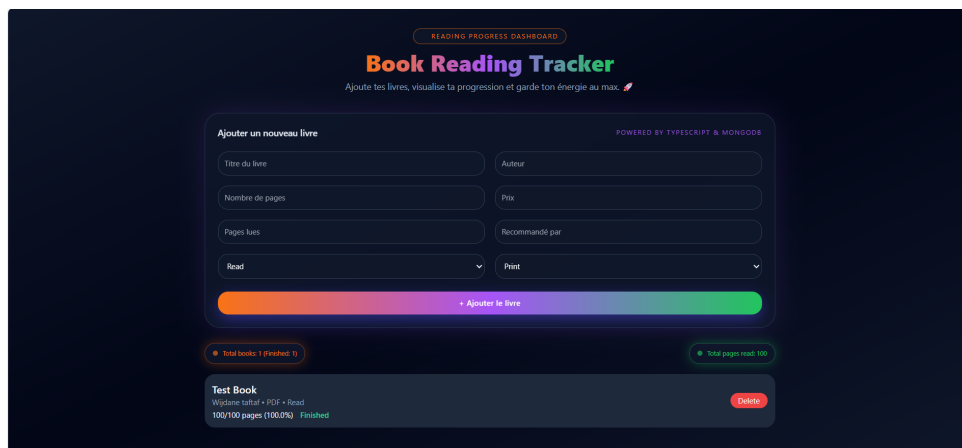


FIGURE 1 – Interface utilisateur de l'application Book Reading Tracker.

```
_id: ObjectId('69ef57ed6674927b316d9dbf')
title: "Test Book"
author: "Wijdane taftaf"
pages: 100
status: "Read"
price: 40
pagesRead: 100
format: "PDF"
suggestedBy: "Prof"
finished: true
__v: 0
```

FIGURE 2 – Enregistrement d'un livre dans la base de données MongoDB.

Conclusion

Ce travail pratique m'a permis de concevoir et de déployer une application Web complète en appliquant concrètement les notions abordées en cours. L'utilisation de TypeScript m'a aidée à structurer le code de manière plus fiable grâce au typage statique, aux classes, aux modules et aux énumérations, tandis que TailwindCSS m'a permis de mettre en place une interface moderne, claire et réactive. Côté backend, l'intégration de Node.js, Express et MongoDB a offert une architecture REST robuste assurant la persistance et la cohérence des données. L'ensemble de la solution illustre l'importance d'une séparation nette entre frontend, backend et base de données, et constitue une base solide pour développer, à l'avenir, des fonctionnalités plus avancées telles que la recherche, la mise à jour des livres ou l'ajout d'un système d'authentification utilisateur.